

Project Overview

The project is a **WPF .NET 9 MVVM image composition editor**. The application allows composing multiple layers (images, text, and compositions), manipulating them interactively, and rendering the result to a 1 bpp image suitable for printing or further processing.

The architecture remains intentionally simple:

- **Models** store data only.
 - **ViewModels** handle interaction.
 - **ImageService** owns all image processing.
 - No dependency injection or unnecessary interfaces.
-

Current Layer Types

ImageLayer

Represents an imported bitmap.

Stores:

- Position
- Rotation
- Scale
- Size
- Center
- Bitmap (`Mat`)
- `TransparentWhite`

The bitmap is always grayscale (`CV_8UC1`).

TextLayer

Stores:

- Text
- Font
- Position
- Rotation
- Scale

Rendering is generated on demand using SkiaSharp.

The bitmap is **not stored**.

CompositionLayer

Represents another saved composition imported into the current composition.

Stores:

- imported grayscale bitmap
- positioning
- transforms
- `TransparentWhite`

Imported compositions preserve anti-aliased grayscale.

ImageService Responsibilities

ImageService owns every image conversion.

It performs:

- image loading
- conversion to grayscale
- conversion to 1 bpp
- transparent preview generation
- importing compositions
- cropping imported compositions
- saving rendered images

Models contain **no image processing code**.

Imported Composition Pipeline

Import now works as follows:

```
Load image
  ↓
Convert to grayscale
  ↓
Find bounding box
(using Compare or Threshold)
  ↓
```



```
Crop grayscale image
    ↓
Store cropped grayscale bitmap
    ↓
If TransparentWhite
    create BGRA preview
else
    keep grayscale preview
```

The original grayscale anti-aliasing is preserved.

Rendering

Rendering creates a white `CV_8UC1` canvas.

Layers are rendered from bottom to top.

ImageLayer

```
WarpAffine
    ↓
TransparentWhite ?
    ↓
yes → create mask from black pixels
    CopyTo(rendered, mask)

no  → CopyTo(rendered)
```



TextLayer

Pipeline:

```
Measure text
    ↓
Create Gray8 SKBitmap
    ↓
Render text with antialiasing
    ↓
Convert to OpenCV Mat
    ↓
WarpAffine
    ↓
Threshold
```





Text always has a transparent white background.

CompositionLayer

Uses exactly the same rendering logic as `ImageLayer`.

```
WarpAffine
  ↓
TransparentWhite ?
  ↓
yes → mask
    CopyTo(rendered, mask)

no  → CopyTo(rendered)
```



Rendering and preview now have consistent transparency semantics.

Transparency Semantics

```
TransparentWhite == true
```

- white pixels are transparent
- black pixels overwrite the destination

```
TransparentWhite == false
```

- every pixel overwrites the destination

The same rule now applies consistently to:

- preview
- rendering
- imported compositions

Image Formats

Internal working format:

```
CV_8UC1
```



Transparent previews:

```
CV_8UC4 (BGRA)
```



created by:

GRAY → BGRA



adding an alpha channel.

OpenCV has no first-class Gray+Alpha format.

Rendering Philosophy

The renderer follows OpenCV's native philosophy:

```
image
+
mask
```



instead of

```
Gray + Alpha
```



Masks determine which pixels participate in compositing.

Current Features

Implemented:

- Load image
- Convert image to 1 bpp
- Import composition
- Preserve grayscale antialiasing
- Crop imported compositions
- Image layers
- Text layers
- Composition layers
- Interactive move
- Scale
- Rotate (90° increments)
- Layer selection
- Layer ordering
- Transparent previews
- Consistent rendering

- Save rendered image
 - Save preview image
-

Architecture Status

The current architecture has stabilized.

The rendering pipeline, import pipeline, transparency handling, and layer model are internally consistent. Future work can now focus on higher-level functionality (project serialization, undo/redo, additional editing features, printing, etc.) rather than further restructuring of the rendering system.